Deep Reinforcement Learning

Monte Carlo Methods

Prof. Dr. Sebastian Peitz

Scientific Machine Learning group @ Paderborn University

Winter term 2026/27

Content

Content

- Monte Carlo simulation
- MC prediction
- MC esitmation of action values
- MC control
- On- and off-policy learning
- Off-policy MC control
- Summary

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1	Multi-armed bandits	Exploration-exploitation dilemma
2	Markov decision processes	Dynamics, rewards, policies
3	Dynamic programming	Optimal decision making with *full knowledge*
4	Monte Carlo methods	*Data-driven learning* from entire episodes
5	Temporal difference learning & Q-learning	
	Deep-learning-based methods	
	Model-Based Control	
	Advanced Topics	

Lecture contents

Monte Carlo simulation

What does *Monte Carlo sampling* mean?



Monte Carlo Port (Source)

- We're trying to estimate the expected value of function of a random variable $x = X(\omega)$:

$$\mathbb{E}\left[f(X)\right] = \sum_{\omega \in \Omega} p(x) f(x) \qquad / \qquad \mathbb{E}\left[f(X)\right] = \int_{\omega \in \Omega} p(x) f(x)\, \mathrm{d}x.$$

- Not knowing f or the random variable X, we need to rely on *experience*.
- Monte Carlo sampling: Having no prior knowledge, we can estimate the expected value from repeated experiments.
- This lecture: **Let's use this for (episodic, i.e., $T < \infty$) RL when we don't know the MDP!**

Example: Estimating the value for π

- The area of a quarter circle with radius $r = 1$:

$$A = \frac{1}{4}\pi r^2 = \frac{\pi}{4}.$$

- Randomly and uniformly place points in the unit square:

$$(x, y) \sim U([0, 1]^2).$$

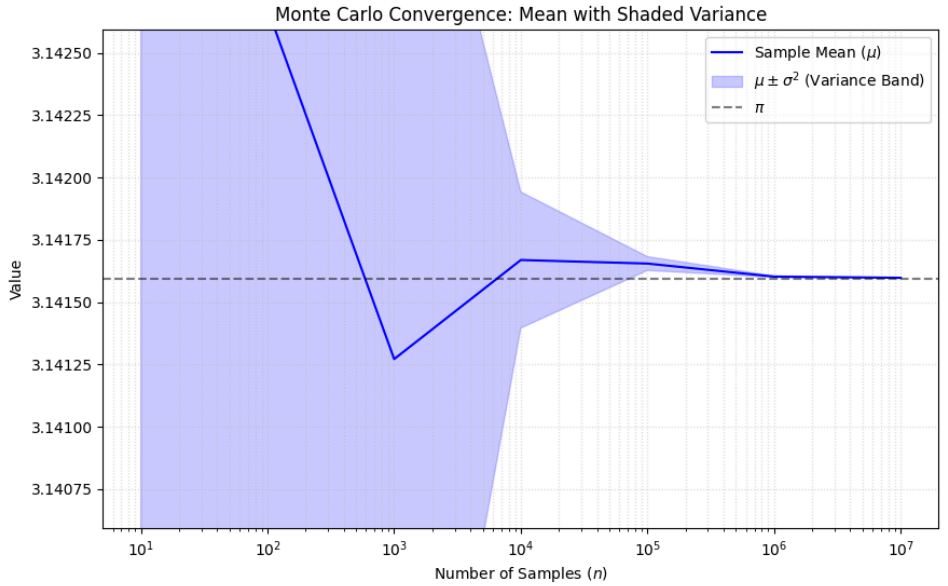
- Probability of landing **inside** the quarter circle is the ratio of the areas:

$$p(x^2 + y^2 \leq 1) = \frac{A}{1^2} = \frac{\pi}{4}.$$

- Monte Carlo estimate:

$$\pi = 4 \cdot \mathbb{E}\left[p(x^2 + y^2 \leq 1)\right].$$

- Let's estimate π with various numbers of samples. Repeated experiments for mean and variance at different n.



Monte Carlo prediction

Monte Carlo prediction

We begin by learning V^π for a given policy π.

- Recall that $V^\pi(s)$ is the expected return (expected cumulative future discounted reward) starting from that state:

$$V(s) = \mathbb{E}\left[g_t | s_t = s\right] = \mathbb{E}\left[\sum_{k=0}^{T} \gamma^k r_{t+k} \middle| s_t = s\right].$$

- Simple way to estimate from experience: average the returns observed after visits to a state s.
- As more returns are observed, the average should converge to the expected value.
- Two different approaches:
  - First-visit MC: approximate the return only from the first visit to a state s.
  - Every-visit MC: calculate an update for the $V(s)$ estimate for every visit to a state s.

Algorithm: First-visit MC prediction for estimating $V \approx V^\pi$.
initialize

- $V(s)$ arbitrarily for $s \in \mathcal{S}$
- $\ell(s)$: an empty list of returns for all $s \in \mathcal{S}$

for $k = 1, 2, \ldots, K$ episodes:
  $g \leftarrow 0$
  Generate a sequence following π:

$$((s_0, a_0, r_0), (s_1, a_1, r_1), \ldots, (s_{T_k-1}, a_{T_k-1}, r_{T_k-1}))$$

  for $t = T_k - 1, T_k - 2, T_k - 3, \ldots, 0$:
    $g \leftarrow \gamma g + r_t$
    if $s_t \notin \{s_0, \ldots, s_{t-1}\}$ **then** (*that's the first-visit condition*)
      Append g to $\ell(s_t)$
      $V(s_t) = \textbf{average}(\ell(s_t))$

Convergence? Yes, for every state $s \in \mathcal{S}$ that is visited infinitely often ($\rightarrow$ exploration!)

MC prediction (every-visit version)

Algorithm: First-visit MC prediction for estimating $V \approx V^\pi$.
initialize

- $V(s)$ arbitrarily for $s \in \mathcal{S}$
- $\ell(s)$: an empty list of returns for all $s \in \mathcal{S}$

for $k = 1, 2, \ldots, K$ episodes:
$g \leftarrow 0$
Generate a sequence following π:

$$((s_0, a_0, r_0), (s_1, a_1, r_1), \ldots, (s_{T_k-1}, a_{T_k-1}, r_{T_k-1}))$$

for $t = T_k - 1, T_k - 2, T_k - 3, \ldots, 0$:
$g \leftarrow \gamma g + r_t$
~~**if** $s_t \notin \{s_0, \ldots, s_{t-1}\}$:~~ (*only difference to first-visit*)
Append g to $\ell(s_t)$
$V(s_t) = \textbf{average}(\ell(s_t))$

Advantages of MC methods

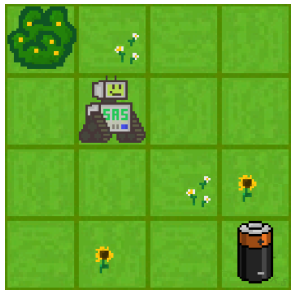
1. We can learn from experience, even if we do not know the model

- actual experience,
- simulated experience, if we have a model (but do not necessarily know the transition probability $p\left(s'|s,a\right)$).

2. The estimates for each state are independent of each other
   $\Rightarrow$ we are not bootstrapping, i.e., building estimates based on other estimates.
3. As a consequence, we can estimate parts of the value function that are of particular interest.

Example: Gridworld (same as in the DP lecture)

We have a small robot in a gridworld that wants to recharge.



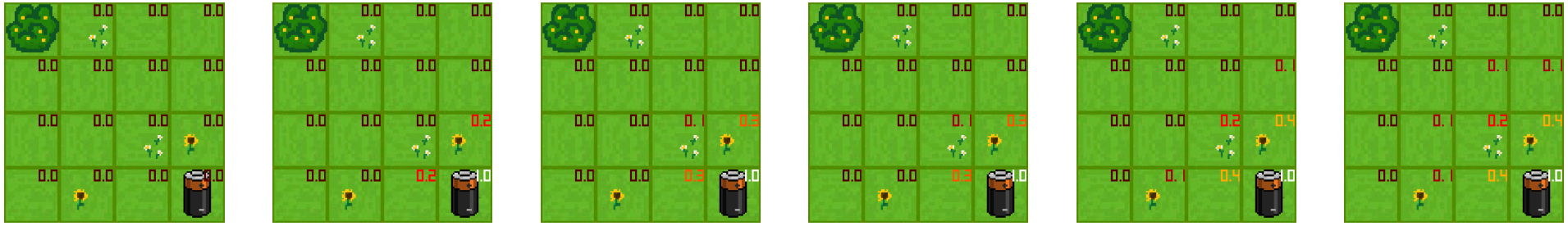
Gridworld



Random policy

- Initial state s_0: a random valid field.
- Goal: reach the battery ($r = 1$, otherwise $r = 0$).
- $\mathcal{A} = \{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ (*leaving* or *invalid field* $\Rightarrow$ no movement).
- $\pi(\cdot|s) = [0.25, 0.25, 0.25, 0.25]^\top \; \forall \; s \in \mathcal{S}$.
- Discount: $\gamma = 0.8$.
- Terminal state: If the robot hits the battery, it receives $r = 1$ (potentially discounted) and the episode ends.
- Optimal strategy: Move towards the battery as quickly as possible.

If we know the model (i.e., p), we can use the *iterative policy evaluation* algorithm:

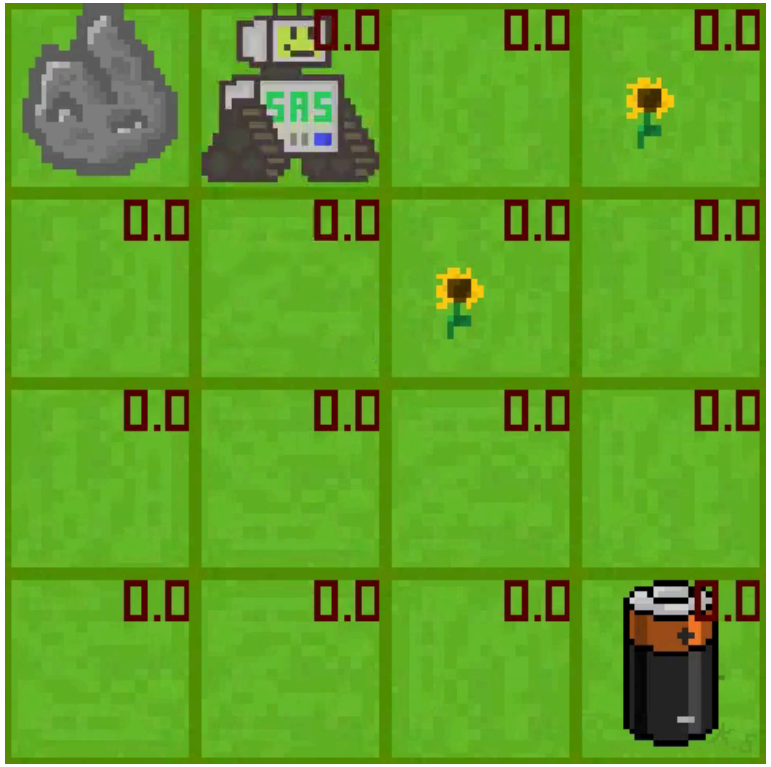


$k = 0$ $k = 1$ $k = 2$ $k = 3$ $k = 10$ $k = 1000$

DP: In-place iterative policy evaluation

Example: Gridworld

Now: Every-visit MC prediction of V^{π}, for $\pi(\cdot|s) = [0.25, 0.25, 0.25, 0.25]^{\top} \; \forall \, s \in \mathcal{S}$.



Monte Carlo estimation of action values

Monte Carlo estimation of action values

Is a model available (i.e., the MDP $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ is known)?

- Yes $\Rightarrow$ state values plus one-step predictions deliver an optimal policy.
- No $\Rightarrow$ action values are very useful to directly obtain optimal policies.

Estimating $Q^\pi(s, a)$ using MC:

- Analog to the MC prediction algorithm for $V^\pi(s)$.
- Small variation: A visit refers to a state-action pair (s, a).
- First-visit and every-visit variants exist

Possible challenges?

- Certain state-action pairs (s, a) may never be visited.
- Missing degree of exploration (in particular for deterministic π).
- Workaraound: **exploring starts** $\Rightarrow$ start episodes in random state-action pairs (s, a).

Monte Carlo control

Monte Carlo control (1)

We learned about the Generalized policy iteration (GPI) in the last lecture.

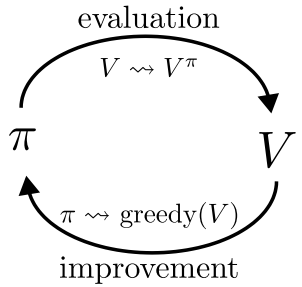
Let's use the same procedure here:

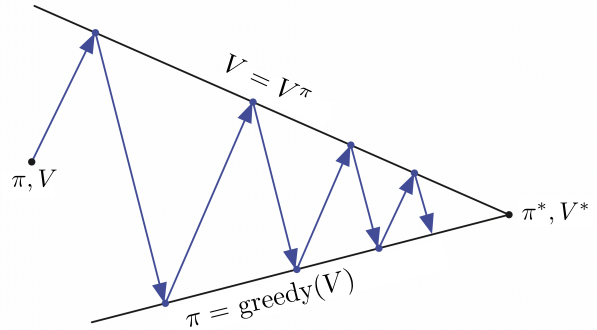
$$\pi_0 \xrightarrow{E} Q^{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} Q^{\pi_1} \dots \xrightarrow{I} \pi^* \xrightarrow{E} Q^* = Q^{\pi^*}.$$

Policy improvement: Make the policy greedy w.r.t. the current value function:

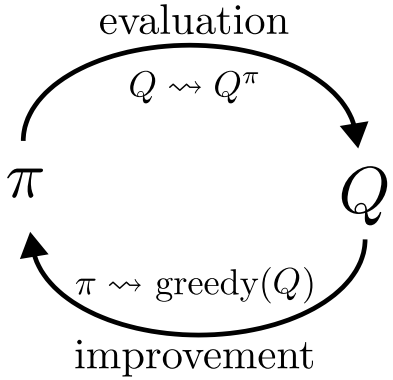
Since we have the *action-value function* Q, we don't need a model to construct the policy:

$$\pi(s) = \arg\max_{a \in \mathcal{A}} Q(s, a).$$





GPI (Sutton and Barto 1998)



GPI for MC control (Sutton and Barto 1998)

Monte Carlo control (2)

Even if we're operating in an unknown MDP, the policy improvement theorem remains valid for MC-based control due to the underlying MDP structure:

Policy improvement theorem for MC-based control

$$
\begin{aligned}
Q^{\pi_k}(s, \pi_{k+1}(s)) &= Q^{\pi_k}\left(s, \arg\max_{a \in \mathcal{A}} Q^{\pi_k}(s, a)\right) \\
&= \max_{a \in \mathcal{A}} Q^{\pi_k}(s, a) \\
&\geq Q^{\pi_k}(s, \pi_k(s)) \\
&\geq V^{\pi_k}(s).
\end{aligned}
$$

- **Policy improvement**: Construct the greedy policy $\pi_{k+1}(s)$ w.r.t. the current approximation Q^{π_k}
- **Assumptions required**:
  - The episodes have epxloring starts. *(We will consider this later)*
  - We are training on an infinite number of episodes.

Monte Carlo control (3)

Algorithm: Monte Carlo ES (Exploring Starts) for estimating $\pi \approx \pi^*$.
initialize

- $\pi(s)$ arbitrarily for $s \in \mathcal{S}$
- $Q(s,a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $\ell(s,a)$: an empty list of returns for all $s \in \mathcal{S}, a \in \mathcal{A}$

for $k = 1, 2, \ldots, K$ episodes *(or until π converges)*:
$g \leftarrow 0$
Choose $s_0 \in \mathcal{S}$ and $a_0 \in \mathcal{A}$ randomly such that all pairs have probability > 0
Generate a sequence starting at (s_0, a_0) and following π:

$$((s_0, a_0, r_0), (s_1, a_1, r_1), \ldots, (s_{T_k-1}, a_{T_k-1}, r_{T_k-1}))$$

for $t = T_k - 1, T_k - 2, T_k - 3, \ldots, 0$:
$g \leftarrow \gamma g + r_t$
if $(s_t, a_t) \notin \{(s_0, a_0), \ldots, (s_{t-1}, a_{t-1})\}$ **then**
Append g to $\ell(s_t)$
$Q(s_t, a_t) = \textbf{average}(\ell(s_t, a_t))$
$\pi(s_t) \leftarrow \arg\max_{a \in \mathcal{A}} Q(s_t, a)$

- It is clear that this algorithm cannot converge to a suboptimal policy
- Stability is only achieved when both π and Q are optimal

Incremental implementation

- The previous algorithm (as well as all other before where we have used averaging) is computationally inefficient!
- Instead, it is better to update the estimate for Q (or V) incrementally.
- The sample mean $\mu_1, \mu_2, \ldots$ of an arbitrary sequence $g_1, g_2, \ldots$ is:

$$\mu_K = \frac{1}{K}\sum_{k=1}^{K} g_k = \frac{1}{K}\left[g_K + \sum_{k=1}^{K-1} g_k\right] = \frac{1}{K}\left[g_K + (K-1)\mu_{K-1}\right] = \mu_{K-1} + \frac{1}{K}\left[g_K - \mu_{K-1}\right].$$

- In terms of the averaging function from the algorithm before, this means that we can update $Q(s_t, a_t)$ incrementally (with $n(s_t, a_t)$ being the number of occurrences of the tuple (s_t, a_t)):

$$Q(s_t, a_t) = \textbf{average}(\ell(s_t, a_t)) \qquad \Rightarrow \qquad Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{1}{n(s_t, a_t)}\left[g - Q(s_t, a_t)\right].$$

💡 if a problem is **non-stationary**, we can use a step size $\alpha \in (0, 1]$ as in the multi-armed bandit setting: $Q(s_t, a_t) = Q(s_t, a_t) + \alpha\left[g - Q(s_t, a_t)\right]$.

Monte Carlo control (4) – incremental implementation

Algorithm: Monte Carlo ES (Exploring Starts) for estimating $\pi \approx \pi^*$.
initialize

- $\pi(s)$ arbitrarily for $s \in \mathcal{S}$
- $Q(s,a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $n(s,a) = 0 \; \forall \; s \in \mathcal{S}, a \in \mathcal{A}$: a list of state-action visits (~~an empty list of returns ℓ~~)

for $k = 1, 2, \ldots, K$ episodes *(or until π converges)*:
$g \leftarrow 0$
Choose $s_0 \in \mathcal{S}$ and $a_0 \in \mathcal{A}$ randomly such that all pairs have probability > 0
Generate a sequence starting at (s_0, a_0) and following π:

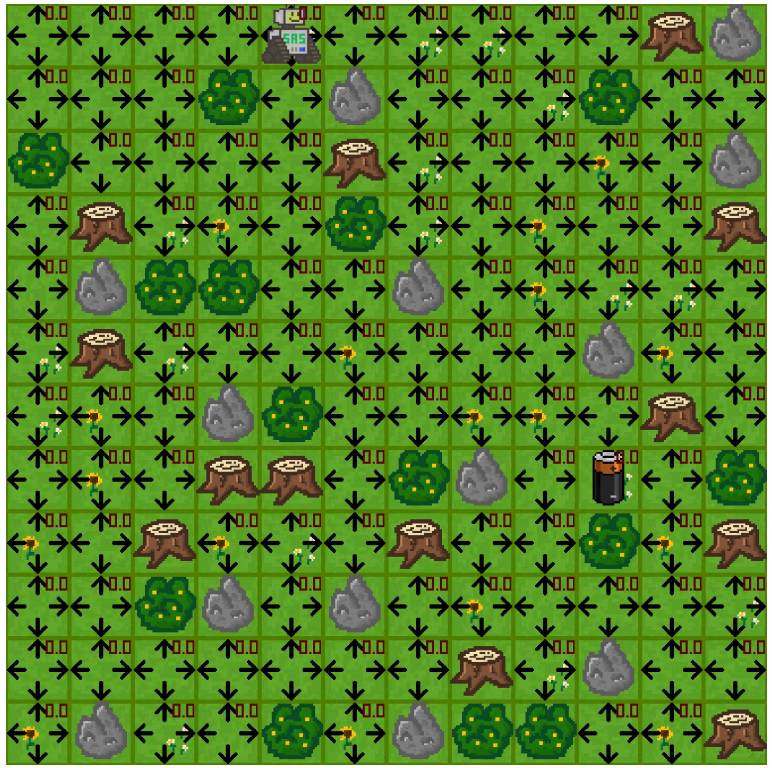
$$((s_0, a_0, r_0), (s_1, a_1, r_1), \ldots, (s_{T_k-1}, a_{T_k-1}, r_{T_k-1}))$$

for $t = T_k - 1, T_k - 2, T_k - 3, \ldots, 0$:
$g \leftarrow \gamma g + r_t$
if $(s_t, a_t) \notin \{(s_0, a_0), \ldots, (s_{t-1}, a_{t-1})\}$ **then**
$n(s_t, a_t) \leftarrow n(s_t, a_t) + 1$ (~~appending g to the list of returns~~)
$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{1}{n(s_t, a_t)}[g - Q(s_t, a_t)]$ (~~averaging over the list of returns~~)
$\pi(s_t) = \arg\max_{a \in \mathcal{A}} Q(s_t, a)$

Example: Gridworld

Let's apply the Monte Carlo Exploring Starts algorithm to two gridworlds of different sizes:





On-policy and off-policy learning

On-policy versus off-policy (1)

On-Policy: "Learning by Doing"

- The agent only learns from what it's currently doing.
- It evaluates and improves the exact same policy it is using to make decisions.
- For example,
  - if it is being cautious, it learns how to improve cautious behavior.
  - if it tries something risky and fails, it learns from that failure but can't easily "imagine" what would have happened if it had taken a different action earlier.

Off-Policy: "Learning by Observing"

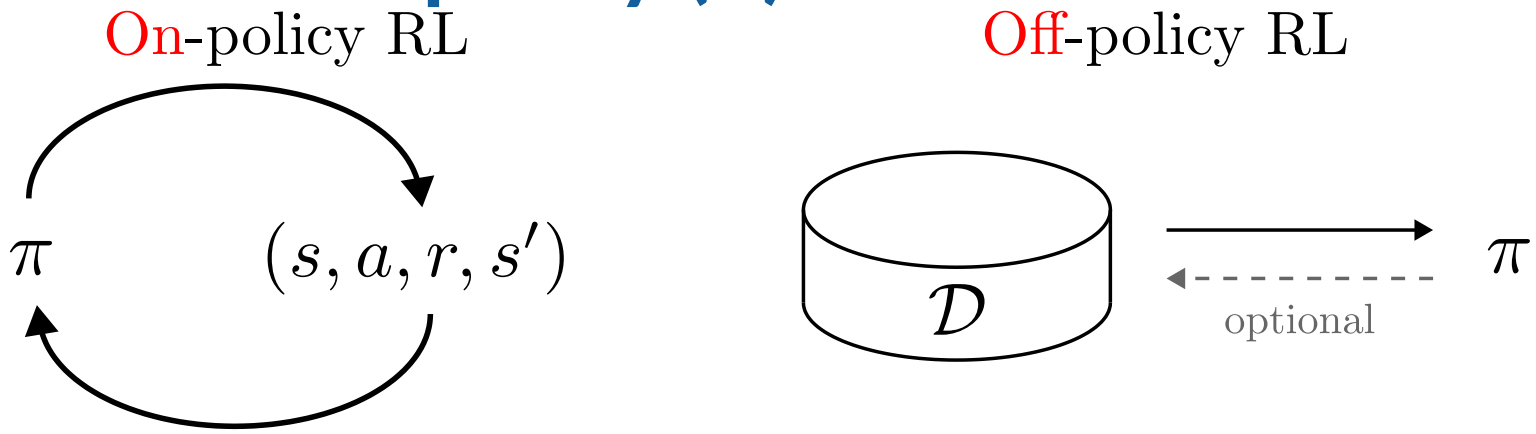
- is more like a student watching an expert or looking at a variety of different actors once.
- It can follow one path (the "behavior policy") while learning the best possible path (the "target policy").
- It separates "how I am acting" from "what I am learning."
- Benefit: It can learn from old data, from human demonstrations, or by watching other agents.

Question: Which class does the *Monte Carlo ES* algorithm belong to?

Answer: It is on-policy! Each time we update our policy (the counter k for the episodes), we collect the returns anew, following the current policy π.

We will cover the topic on-policy versus off-policy in more detail later in this course.

On-policy versus off-policy (2)



Feature	On-policy	Off-policy
Learning Source	Learns from its current actions only.	Can learn from any data (past, random, or expert).
Exploration	Often more stable but can get stuck in “safe” habits.	More aggressive at finding the absolute best strategy.
Efficiency	Lower. Throws away data once the policy changes.	Higher. Can “re-use” old experiences (Experience Replay).

Key differences at a glance:

ϵ-greedy as an on-policy alternative (1)

Motivating question: How do we get rid of the restrictive (and often not achievable) requirement of *exploring starts (ES)*?

- We need to make sure that we visit all state-action pairs, irrespective of where we start!
- Exploration requirement:

$$\pi(a|s) > 0 \quad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

- Policies of this type are called **soft**.
- The level of exploration should be tunable during the learning process.

Where have we seen something like this before? $\Rightarrow$ In the multi-armed bandits lecture!

ϵ-greedy on-policy learning

- With probability ϵ, the agent's choice (i.e., the policy output) is overwritten by a random action.
- Probability of all non-greedy actions: $\frac{\epsilon}{|\mathcal{A}|}$.
- Probability of the greedy action: $1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|}$.

ϵ-greedy as an on-policy alternative (2)

Algorithm: ϵ-greedy on-policy first-visit MC control.
initialize

- $\pi(s)$ arbitrarily for $s \in \mathcal{S}$
- $Q(s,a)$ arbitrarily for $s \in \mathcal{S}, a \in \mathcal{A}$
- $n(s,a) = 0 \; \forall \; s \in \mathcal{S}, a \in \mathcal{A}$: a list of state-action visits

for $k = 1, 2, \ldots, K$ episodes *(or until π converges)*:

$g \leftarrow 0$

Choose $s_0 \in \mathcal{S}$ and $a_0 \in \mathcal{A}$ randomly such that all pairs have probability > 0

Generate a sequence $\{(s_t, a_t, r_t)\}_{t=1}^{T_k}$ starting at (s_0, a_0) and following π

for $t = T_k - 1, T_k - 2, T_k - 3, \ldots, 0$:

$g \leftarrow \gamma g + r_t$

if $(s_t, a_t) \notin \{(s_0, a_0), \ldots, (s_{t-1}, a_{t-1})\}$ **then**

$n(s_t, a_t) \leftarrow n(s_t, a_t) + 1$

$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{1}{n(s_t, a_t)} [g - Q(s_t, a_t)]$

$\tilde{a} = \arg\max_{a \in \mathcal{A}} Q(s_t, a)$

$$\pi(a|s_t) = \begin{cases} 1 - \epsilon + \epsilon / |\mathcal{A}|, & a = \tilde{a} \\ \epsilon / |\mathcal{A}|, & a \neq \tilde{a} \end{cases}$$

ϵ-greedy policy improvement

Theorem: Policy improvement for ϵ-greedy policies
Given an MDP, an ϵ-greedy policy π' w.r.t. Q^π is an improvemnt over the ϵ-soft policy π, i.e., $V^{\pi'} > V^\pi$ for all $s \in \mathcal{S}$.

Proof:

$$
\begin{aligned}
V^{\pi'}(s) = Q^\pi(s, \pi'(s)) &= \sum_{a \in \mathcal{A}} \pi'(a|s) Q^\pi(s,a) \\
&= \frac{\epsilon}{|\mathcal{A}|} \left(\sum_{a \in \mathcal{A}} Q^\pi(s,a) \right) + (1-\epsilon) \max_{a \in \mathcal{A}} Q^\pi(s,a) \\
&\geq \frac{\epsilon}{|\mathcal{A}|} \left(\sum_{a \in \mathcal{A}} Q^\pi(s,a) \right) + (1-\epsilon) \left(\sum_{a \in \mathcal{A}} \frac{\pi(a|s) - \frac{\epsilon}{|\mathcal{A}|}}{1-\epsilon} Q^\pi(s,a) \right)
\end{aligned}
$$

(Last term: weighted average over all actions. It thus has to be smaller than the max operation.)

$$
\begin{aligned}
&= \frac{\epsilon}{|\mathcal{A}|} \left(\sum_{a \in \mathcal{A}} Q^\pi(s,a) \right) - \frac{\epsilon}{|\mathcal{A}|} \left(\sum_{a \in \mathcal{A}} Q^\pi(s,a) \right) + \sum_{a \in \mathcal{A}} \pi(a|s) Q^\pi(s,a) \\
&= V^\pi(s).
\end{aligned}
$$

Greedy in the limit with infinite exploration (GLIE)

Definition: Greedy in the limit with infinite exploration (GLIE)

A learning policy π is called GLIE if it satisfies the following two properties:

- If a state is visited infinitely often, then each action is chosen infinitely often:

$$\lim_{k \to \infty} \pi_k(a|s) = 1 \quad \forall\, s \in \mathcal{S}, a \in \mathcal{A}.$$

- In the limit ($i \to \infty$) the learning policy is greedy with respect to the learned action value:

$$\lim_{k \to \infty} \pi_k(a|s) = \pi(s) = \arg\max_{a \in \mathcal{A}} Q^{\pi}(s, a) \quad \forall s \in \mathcal{S}.$$

GLIE Monte Carlo control

Theorem: Optimal decision using MC-control with ϵ-greedy

MC-based control using ϵ-greedy exploration is GLIE, if ϵ is decreased at rate

$$\epsilon_k = \frac{1}{k},$$

with k being the increasing episode index. In this case,

$$Q^{\pi}(s,a) = Q^{*}(s,a).$$

Remarks:

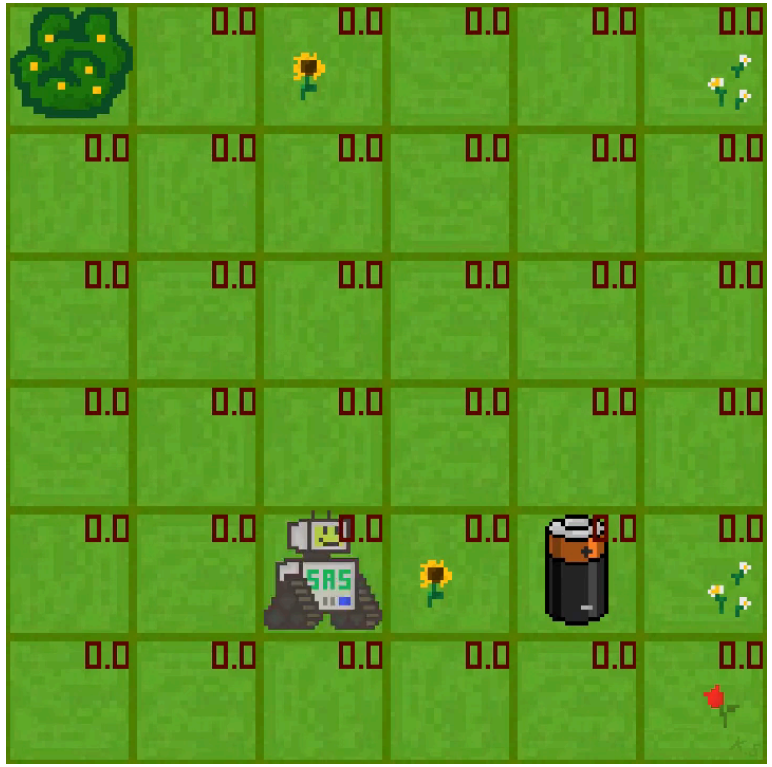
- Limited feasibility: infinite number of episodes required.
- ϵ-greedy is an undirected and unmonitored random exploration strategy. Can that be the most efficient way of learning?

Example: Gridworld - fixed starting point

Fixed-start, every-visit action-value estimation:



$\epsilon = 0$



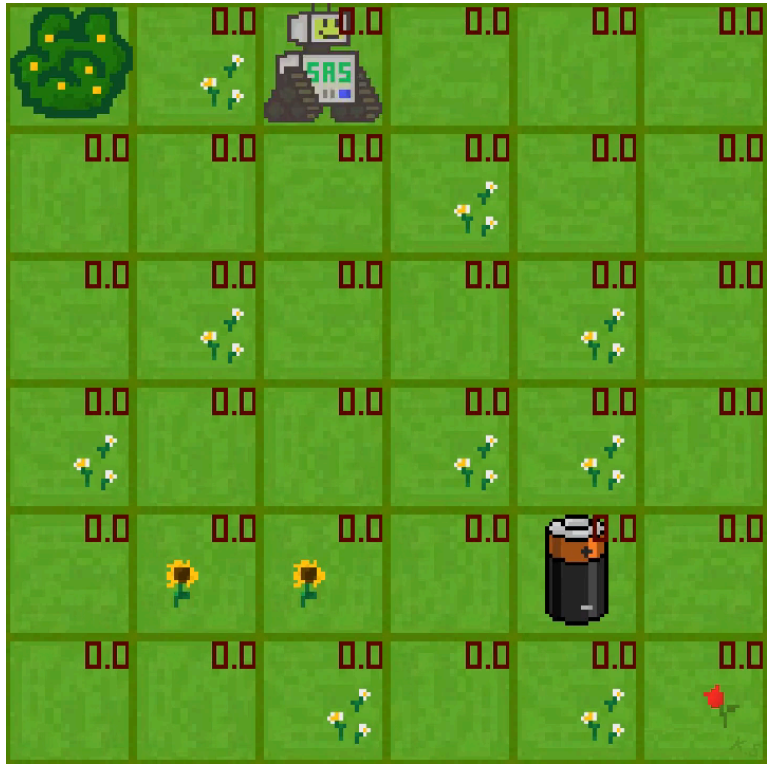
$\epsilon = 0.2$

Example: Gridworld - exploring starts

Exploting starts (ES), every-visit action-value estimation:



$\epsilon = 0$



$\epsilon = 0.2$

Off-policy Monte Carlo control

Off-policy learning background

All learning control methods face a dilemma:

- They seek to learn action values conditional on subsequent optimal behavior,
- but they need to behave non-optimally in order to explore all actions.
- The previous ϵ-greedy approaches were only a compromise: learning action values for *near-optimal* policies.

The off-policy learning idea:

- Use two policies:
  - **Behavior policy** $\pi_\beta(a|s)$: exploratory, used to generate behavior.
  - **Target policy** $\pi(a|s)$: learns from that experience to become the optimal policy.
- Use cases:
  - Learn from observing humans or other agents/controllers.
  - Re-use experience generated from old policies ($\pi_0, \pi_1, \ldots$).
  - Learn about multiple policies while following one policy.

Off-policy prediction problem statement

MC off-policy prediction problem statement.

- Both policies are considered fixed (that's the *prediction* assumption).
- Estimate V^π or Q^π while following $\pi_\beta(a|s)$.

Requirement:

- **Coverage**: every action taken under π must be (at least occasionally) taken under π_β as well.

$$\pi(a|s) > 0 \quad \Rightarrow \quad \pi_\beta(a|s) > 0 \qquad \forall s \in \mathcal{S}, a \in \mathcal{A}.$$

- Consequence:
  - In any state where $\pi_\beta \neq \pi$, π_β has to be a stochastic policy.
  - Nevertheless, π may be either stochastic or deterministic.

Importance sampling (1)

Example

- We're trying to calculate the average height of people in a city, but only have access to a guest list for a professional basketball convention.
- Taking the average of this sample will likely lead to a too large value.
- Importance sampling: mathematical *correction* to use data from one distribution to estimate properties of another distribution.



The central concept: re-weighting

- We want to know the expected value under a target distribution $\pi(s)$, but only have samples from another distribution $\pi_\beta(s)$.
- We calculate the **importance weight**:

$$w = \frac{\pi(s)}{\pi_\beta(s)}$$

- If a sample is common in π but rare in π_β, we give it a high weight (it is *important*).
- If a sample is rare in π but common in π_β, we give it a low weight.

Caveat: if w is large (distinctly different policies) the estimate's variance is large (i.e., *uncertain* for small numbers of samples).

Importance sampling (2)

Given a starting state s_t, the probability of the subsequent state–action trajectory $(a_t, s_{t+1}, a_{t+1}, \dots, s_T)$ under a policy π is

$$\begin{aligned} p\left(a_t, s_{t+1}, a_{t+1}, \dots, s_T | s_t, \pi\right) &= \pi\left(a_t | s_t\right) p\left(s_{t+1} | s_t, a_t\right) \pi\left(a_{t+1} | s_{t+1}\right) p\left(s_{t+2} | s_{t+1}, a_{t+1}\right) \dots p\left(s_T | s_{T-1}, a_{T-1}\right) \\ &= \prod_{k=t}^{T-1} \pi\left(a_k | s_k\right) p\left(s_{k+1} | s_k, a_k\right). \end{aligned}$$

Importance sampling ratio

The relative probability of a trajectory under the target and behavior policy, the importance sampling ratio, from sample step k to T is

$$\rho_{t:T} = \frac{\prod_{k=t}^{T-1} \pi\left(a_k | s_k\right) p\left(s_{k+1} | s_k, a_k\right)}{\prod_{k=t}^{T-1} \pi_\beta(a_k | s_k) p\left(s_{k+1} | s_k, a_k\right)} = \frac{\prod_{k=t}^{T-1} \pi\left(a_k | s_k\right) \cancel{p\left(s_{k+1} | s_k, a_k\right)}}{\prod_{k=t}^{T-1} \pi_\beta(a_k | s_k) \cancel{p\left(s_{k+1} | s_k, a_k\right)}} = \frac{\prod_{k=t}^{T-1} \pi\left(a_k | s_k\right)}{\prod_{k=t}^{T-1} \pi_\beta(a_k | s_k)}.$$

💡 Although the trajectory probabilities depend on the MDP's transition probabilities p (which are generally unknown), the importance sampling ratio ends up depending only on the two policies and the sequence, not on the MDP.

Importance sampling for MC prediction

- We wish to estimate the expected returns g_t under the target policy π, but we only have g_t due to the behavior policy π_β.
- These returns have the wrong expectation:

$$\mathbb{E}\left[g_t | s_t = s\right] = V^\beta(s).$$

- Importance sampling:

$$\mathbb{E}\left[\rho_{k:T}\, g_t | s_t = s\right] = V^\pi(s).$$

OIS: State-value estimation via Monte Carlo ordinary importance sampling
Estimating the state value V^π following a behavior policy b using **ordinary importance sampling** (**OIS**) results in scaling and averaging the sampled returns by the importance sampling ratio per episode:

$$V^\pi(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{k:T(t)}\, g_t}{|\mathcal{T}(s)|}. \qquad \text{(OIS)}$$

Here $\mathcal{T}(s)$ is the set of all time steps in which s is visited, and $T(t)$ is the termination of the episode starting at t.

Weighted importance sampling

- OIS can be shown to be unbiased (Sutton and Barto 1998), meaning that the expectation of (OIS) is always V^π.
- On the other hand, it can be extreme.

WIS: State-value estimation via Monte Carlo importance sampling

Estimating the state value V^π following π_β using **weighted importance sampling** (**WIS**) results in a slightly different scaling:

$$V^\pi(s) = \frac{\sum_{t \in \mathcal{T}(s)} \rho_{k:T(t)}\, g_t}{\sum_{t \in \mathcal{T}(s)} \rho_{k:T(t)}}. \tag{WIS}$$

Main differences between the two (first-visit) versions:

- Bias:
  - (OIS) is unbiased.
  - (WIS) is biased (though it converges asymptotically to zero).
- Variance:
  - The variance of (OIS) is in general unbounded (the variance of the ratios can be unbounded).
  - For (WIS), the largest weight on any single return is one.
  - Assuming bounded returns, the variance of (WIS) converges to zero even if the variance of the ratios themselves is infinite (Precup, Sutton, and Dasgupta 2001)
  - In practice, (WIS) has dramatically lower variance and is strongly preferred.

WIS: incremental implementation

If we want to perform weighted importance sampling in an in an incremental fashion, we need to keep track of all the weights in (WIS). Let's do this exemplary for the value function V_k at iteration k:

$$V_k = \frac{\sum_{t=1}^{n} w_t g_t}{\sum_{t=1}^{n} w_t}, \qquad \text{where} \quad w_t = \rho_{k:T(t)}.$$

In addition to keeping track of V_k, we must maintain for each state the cumulative sum c_k of the weights given to the first k returns. The update rule for V_k is then:

$$V_{k+1} = V_k + \frac{w_k}{c_k}[g_k - V_k] \qquad \text{and} \qquad c_{k+1} = c_k + w_{k+1},$$

with $c_0 = 0$.

Off-policy Monte Carlo control (1)

Algorithm: Off-policy MC prediction (policy evaluation) for $Q \approx Q^*$.
initialize (for all $s \in \mathcal{S}, a \in \mathcal{A}$)

- $Q(s,a)$ arbitrarily
- $c(s,a) \leftarrow 0$

for $k = 1, 2, \ldots, K$ episodes *(or until π converges)*:
$g \leftarrow 0, w \leftarrow 1$
Generate $\{(s_t, a_t, r_t)\}_{t=0}^{T_k}$ following a *soft policy* b
for $t = T_k - 1, T_k - 2, T_k - 3, \ldots, 0$:
$g \leftarrow \gamma g + r_t$
$c(s_t, a_t) \leftarrow c(s_t, a_t) + w$
$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{w}{c(s_t, a_t)}[g - Q(s_t, a_t)]$
$w \leftarrow w \frac{\pi(a_t|s_t)}{\pi_\beta(a_t|s_t)}$

Variation: On-policy

- $\pi_\beta = \pi$.
- $\frac{\pi(a_t|s_t)}{\pi_\beta(a_t|s_t)} = 1$.
- $w = 1$.
- $c(s,a)$ becomves the counter for the number of visits (i.e., $n(s,a)$).

Off-policy Monte Carlo control (2)

We now have our final algorithm that includes everything we have learned so far

Algorithm: Off-policy MC control for estimating $Q \approx Q^*$ and $\pi \approx \pi^*$.
initialize (for all $s \in \mathcal{S}$, $a \in \mathcal{A}$)

- $Q(s,a)$ arbitrarily
- $c(s,a) \leftarrow 0$
- $\pi(s) = \arg\max_{a \in \mathcal{A}} Q(s,a)$ (with ties broken consistently)

for $k = 1, 2, \ldots, K$ episodes *(or until π converges)*:
  $g \leftarrow 0$, $w \leftarrow 1$
  Generate $\{(s_t, a_t, r_t)\}_{t=0}^{T_k}$ following a *soft policy* b
  for $t = T_k - 1, T_k - 2, T_k - 3, \ldots, 0$:
    $g = \gamma g + r_t$
    $c(s_t, a_t) \leftarrow c(s_t, a_t) + w$
    $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \frac{w}{c(s_t, a_t)} [g - Q(s_t, a_t)]$
    $\pi(s_t) \leftarrow \arg\max_{a \in \mathcal{A}} Q(s_t, a)$ (with ties broken consistently)
    if $\pi(s_t) \neq a_t$ **then**
      Exit inner loop and proceed to next episode
    $w \leftarrow w \frac{1}{\pi_\beta(a_t | s_t)}$

Summary / what you have learned

Summary / what you have learned

- MC methods allow **model-free learning of value functions and optimal policies** from experience (i.e., sampled episodes).
- Using full episodes, MC is largely **based on averaging returns**.
- MC-based control **reuses generalized policy iteration (GPI)**, i.e., mixing policy evaluation and improvement.
- Maintaining **sufficient exploration is important**:
  - *Exploring starts*: not feasible in all applications but simple.
  - *On-policy ϵ-greedy learning*: trade-off between exploitation and exploration cannot be resolved easily.
  - *Off-policy learning*: agent learns about a (possibly deterministic) target policy from an exploratory, soft behavior policy.
- **Importance sampling** transforms expectations from the behavior to the target policy.
  - This estimation task comes with a bias-variance-dilemma.
  - Slow learning can result from ineffective experience usage in MC methods.

References

Precup, Doina, Richard S Sutton, and Sanjoy Dasgupta. 2001. “Off-Policy Temporal-Difference Learning with Function Approximation.” In *ICML*, 417–24.

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.